LINKED LIST
PROBLEM 8

QUESTION:

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 38

Question description

Malar, subash, and Yasir alias Pari are three first-year engineering students of the State Technical Institution (STI), India. While Malar and subash are average students who come from a Middle class, Yasir is from a rich family. Malar studies, engineering as per his father's wishes, while subash, whose family is poor, studies engineering to improve his family's financial situation.

Yasir, however, studies engineering of his simple passion for developing android applications.

Yasir is participating in a hackathon for android application development. the task is foling the linked list and also need to follow the given procedure.

- Firstly, Initialize a linked list.
- Then find the middle point using two pointers at different speeds through the sequence of values until they both have equal values.
- Divide the linked list into two halves using these pointer named as slow and fast, then find middle.
- Now reverse the second half of the linked list.
- At last alternate merge of first and second halves.
- Print the fold linked list.

Constraints

0<N<100

0<arr<1000

Input Format

First line indicates the number of elements N to be inserted in array

Second line indicates the array elements according to the N

Output Format

First line represents the d linked list in original order

Second Line represents the linked list in after folding

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
```

EXPECTED OUTPUT

```
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98
17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11
23 13 19 14 18
```

Test Case 2

INPUT (STDIN)

```
15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
```

EXPECTED OUTPUT

```
Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2
0
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
void print(struct node *head)
```

Test Case 3

KEYWORD

```
struct node *next;
```

Test Case 4

KEYWORD

```
void create(struct node
**head,int data)
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	510	105

SOURCE CODE:

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

void create(struct node **head, int data) {
    struct node *new_node = (struct node*)malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = NULL;
    if (*head == NULL) {
        *head = new_node;
    } else {
        struct node *temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = new_node;
    }
}

void print(struct node *head) {
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

struct node* reverseList(struct node* head) {
    struct node *prev = NULL, *curr = head, *next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

void foldList(struct node **head) {
    if (*head == NULL || (*head)->next == NULL) return;
    struct node *slow = *head, *fast = *head;
    while (fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    struct node *head2 = slow->next;
    slow->next = NULL;
    head2 = reverseList(head2);

    struct node *p1 = *head, *p2 = head2;
    while (p2 != NULL) {
        struct node *next1 = p1->next;
        struct node *next2 = p2->next;
        p1->next = p2;
        p2->next = next1;
        p1 = next1;
        p2 = next2;
    }
}

int main() {
    int n, val;
    if (scanf("%d", &n) != 1) return 0;
    struct node *head = NULL;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        create(&head, val);
    }
    printf("Link list data:");
    print(head);

    foldList(&head);
    printf("Link list data after fold:");
    print(head);
    return 0;
}
```

OUTPUT:

```
16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18
```

```
15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2 0
```